

PACKET CAPTURE: PACKET GENERATION

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심
MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergent Network Lab.
<http://mmcn.ajou.ac.kr>



Contents

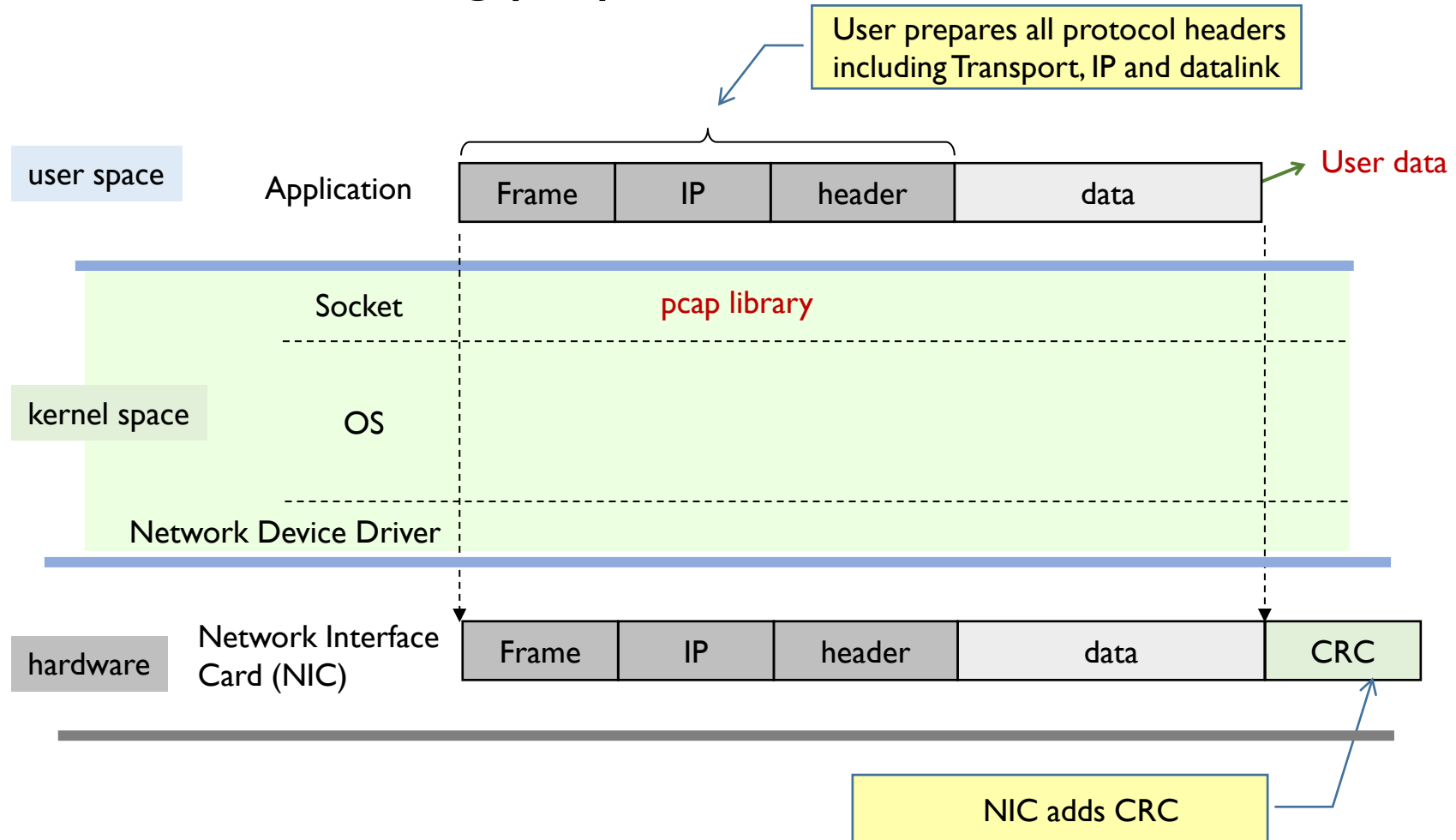
- ❖ Libpcap/WinPcap Packet Generation Functions
- ❖ libnet
- ❖ Linux PF_SOCKET
- ❖ Libpcap Packet Generation Example
- ❖ ARP Example



LIBPCAP/WINPCAP PACKET GENERATION FUNCTIONS

Packet Generation

❖ Packet Generation using pcap



Packet Generation – pcap_inject()

❖ pcap_inject()

- for libpcap
- to send a raw packet through the network interface

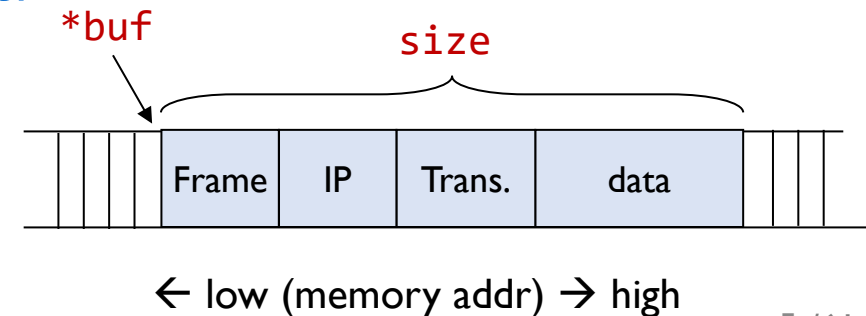
```
int  pcap_inject ( pcap_t      *p,  
                  const void  *buf,  
                  size_t      size);
```

▪ arguments

- **buf** – pointer to the packet including the link-layer header
- **size** – the number of bytes in the packet.

▪ return value

- the number of bytes written on success
- -1 on failure



Packet Generation – pcap_inject()

❖ Example

```
pcap_t *handle;

// Retrieve the device list
pcap_findalldevs (&alldevs, errbuf);

// Open the adapter
handle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);

// Prepare a packet
const char packet[] = { /* packet data*/ };

int bytes_sent = pcap_inject(handle, packet, sizeof(packet));

if (bytes_sent == -1) {
    fprintf(stderr, "Error sending packet: %s\n", pcap_geterr(handle));
} else {
    printf("Sent %d bytes... success !!! \n", bytes_sent);
}
pcap_close(handle);
```

Packet Generation – pcap_sendpacket()

❖ pcap_sendpacket()

- to send a raw packet through the network interface

```
int    pcap_sendpacket (    pcap_t    *p,  
                           const u_char *buf,  
                           int         size);
```

- arguments
 - `buf` – pointer to the packet including the link-layer header
 - `size` – the number of bytes in the packet.
- return value
 - `0` on success
 - only the difference from `pcap_inject( )`
 - `PCAP_ERROR (-1)` on failure

Packet Generation – pcap_sendpacket()

❖ Example

```
pcap_t *handle;

// Retrieve the device list
pcap_findalldevs (&alldevs, errbuf);

// Open the adapter
handle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);

// Prepare a packet
const char packet[] = { /* packet data*/ };

int result = pcap_sendpacket(handle, (const u_char *)packet, sizeof(packet));
if (result != 0 ) {
    fprintf(stderr, "Error sending packet: %s\n", pcap_geterr(handle));
} else {
    printf("Packet sent success !!! \n");
}
pcap_close(handle);
```


Packet Generation – Compariosn

❖ pcap_inject vs. pcap_sendpacket()

Feature	pcap_inject	pcap_sendpacket
Return Value	Number of bytes sent	Success/failure status (0/-1)
Packet Size Type	size_t	int
Success/Failure Check	Can indirectly verify through bytes sent	Directly indicates success or failure
Use Case	When the exact number of bytes sent needs to be verified	Used for simple packet transmission
Compatibility	Mostly used in standard libpcap environments like Linux	More common in Windows/WinPcap environments
Size limitation	No limitation to packet size upto size_t (but, depends on MTU, about 1500 bytes)	Since the packet size is passed as **int** , there may be restrictions when sending large packets (up to 2GB or more).

Packet Generation – Queue Management

❖ Other related functions

- to allocate a send queue

```
pcap_send_queue* pcap_sendqueue_alloc ( u_int memsize );
```

- to add a packet at the end of the send queue

```
int pcap_sendqueue_queue ( pcap_send_queue* queue,  
                           const struct pcap_pkthdr* pkt_header,  
                           const u_char* pkt_data);
```

- to send a queue of raw packets to the network

```
u_int pcap_sendqueue_transmit ( pcap_t* p,  
                                pcap_send_queue* queue,  
                                int sync);
```

- to destroy a send queue

```
void pcap_sendqueue_destroy ( pcap_send_queue* queue );
```

Packet Generation – Example

❖ npcap example – sendpack.c

- in Linux, change the line 131 as

```
#ifdef _WIN32
    *(u_long *)(packet + 14 + 12) = ((struct sockaddr_in *)(addr->addr))->sin_addr.S_un.S_addr;
#else
    *(u_long *)(packet + 14 + 12) = ((struct sockaddr_in *)(addr->addr))->sin_addr.s_addr;;
#endif
```

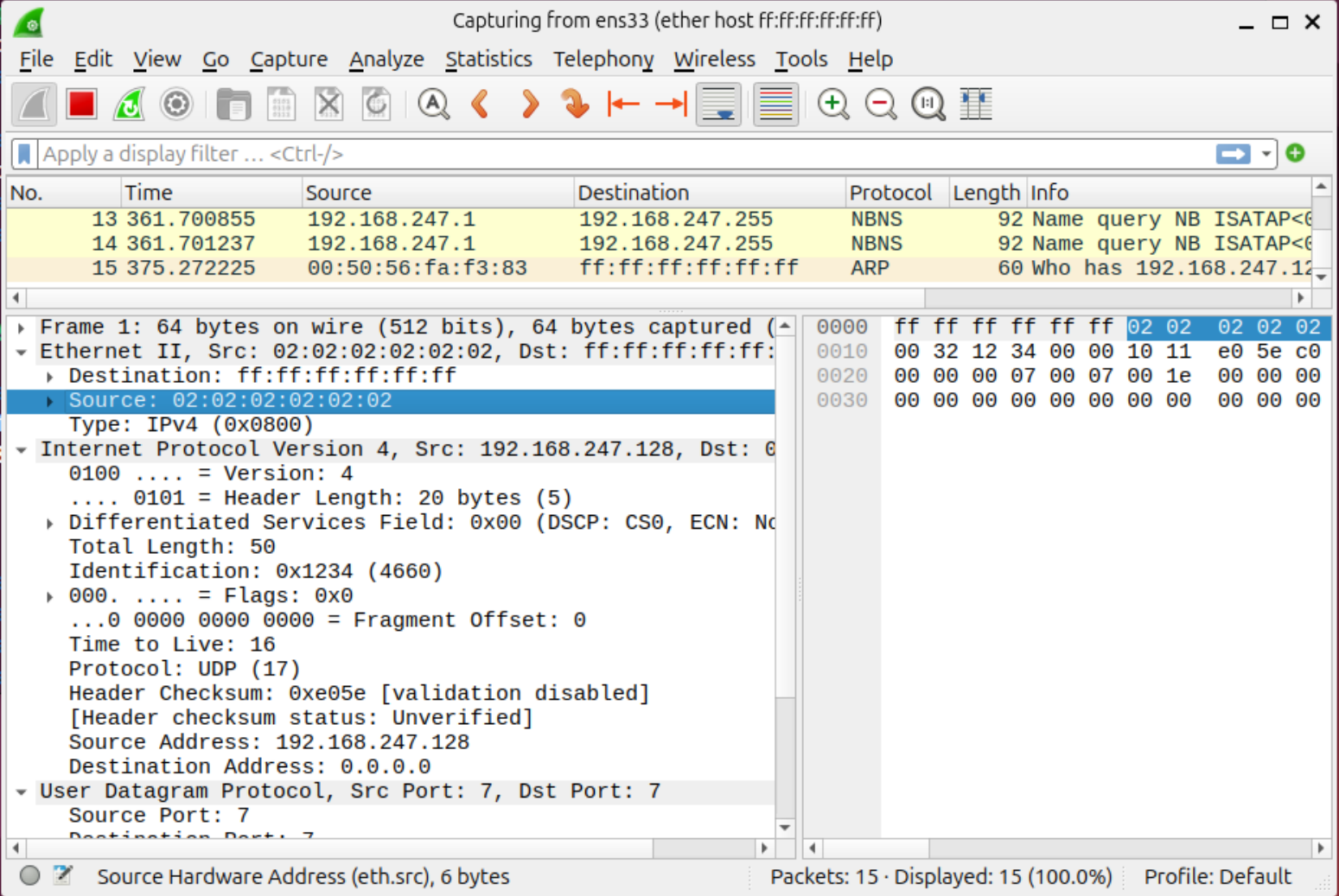
- usage
 - `printf("usage: %s <interface>", argv[0]);`

❖ Prctice

- add routines to select interface from the list of all device interfaces

Packet Generation – Example

```
$ gcc sendpack.c -lpcap
$ sudo ./a.out ens33
```



Capturing from ens33 (ether host ff:ff:ff:ff:ff:ff)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
13	361.700855	192.168.247.1	192.168.247.255	NBNS	92	Name query NB ISATAP<6
14	361.701237	192.168.247.1	192.168.247.255	NBNS	92	Name query NB ISATAP<6
15	375.272225	00:50:56:fa:f3:83	ff:ff:ff:ff:ff:ff	ARP	60	Who has 192.168.247.12

Frame 1: 64 bytes on wire (512 bits), 64 bytes captured (512 bits) on interface 0

Ethernet II, Src: 02:02:02:02:02:02, Dst: ff:ff:ff:ff:ff:ff

- Destination: ff:ff:ff:ff:ff:ff
- Source: 02:02:02:02:02:02
- Type: IPv4 (0x0800)

Internet Protocol Version 4, Src: 192.168.247.128, Dst: 0.0.0.0

- Version: 4
- Header Length: 20 bytes (5)
- Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
- Total Length: 50
- Identification: 0x1234 (4660)
- Flags: 0x0
- Fragment Offset: 0
- Time to Live: 16
- Protocol: UDP (17)
- Header Checksum: 0xe05e [validation disabled] [Header checksum status: Unverified]
- Source Address: 192.168.247.128
- Destination Address: 0.0.0.0

User Datagram Protocol, Src Port: 7, Dst Port: 7

Source Hardware Address (eth.src), 6 bytes

Packets: 15 · Displayed: 15 (100.0%) Profile: Default



LIBNET

libnet



❖ libnet

- <https://github.com/libnet/libnet>
- a C library that simplifies
 - the creation, manipulation, and transmission of network packets
- primarily used for low-level network programming
 - making it easier for tasks like penetration testing, network simulation, and custom protocol creation
 - providing a high-level API to construct packets for various network protocols,
 - such as TCP, UDP, ICMP, ARP, user-customized protocols, and more
 - with each control over packet structure and content
 - with handles the complexities of packet crafting and allows easy control over packet structure and content.

❖ Benefit of libnet

- complex network packet generation works
 - can be handled with simple function calls
- easily combine multiple protocols
 - to create multi-protocol packets
- can be used for various purposes
 - such as network analysis, testing, and attack simulation

❖ Limitation of libnet

- Due to the nature of low-level programming
 - it may be difficult for beginners to use
- There may be platform-specific dependencies
 - Some features may be limited for newer network protocols
- It is ideally targeted for packet generation and transmission
 - not providing packet capture



Install libnet

❖ For linux

```
sudo apt-get install libnet-dev
```

❖ For macOS

```
brew install libnet
```

❖ For Windows

- It can write program with Npcap utilizing libnet source code

libnet programming

❖ basic programming procedure using libnet (1/2)

- initialize libnet

```
l = libnet_init(LIBNET_RAW4, NULL, errbuf);
```

- create a packet (e.g. TCP)

```
u_long dest_ip = libnet_name2addr4(l, "192.168.1.1", LIBNET_RESOLVE);  
// Build TCP packet  
libnet_build_tcp(  
    12345,                // Source Port  
    80,                   // Destination Port  
    0x10000000,           // Sequence Number  
    0x20000000,           // Acknowledgment Number  
    TH_SYN,               // Flags (SYN)  
    32767,                // Window Size  
    0,                    // Checksum (0 for auto-calculation)  
    0,                    // Urgent Pointer  
    LIBNET_TCP_H,         // Total Length  
    NULL,                 // Payload  
    0,                    // Payload Size  
    1,                    // Libnet context  
    0,                    // Protocol Tag  
);
```



libnet programming

❖ basic programming procedure using libnet (2/2)

- send packet

```
res= libnet_write(l)
if ( res == -1)
    fprintf(stderr, "libnet_write() failed: %s\n", libnet_geterror(l));
```

- clean up

```
libnet_destroy(l);
```

- compile

```
gcc <program file> -lnet
```

Sample

❖ icmp_echo_cq.c

- https://github.com/libnet/libnet/blob/master/sample/icmp_echo_cq.c

```
$ gcc icmp_echo_cq.c -lnet
$ sudo ./a.out -s 192.168.0.10 -d 192.168.0.1
libnet 1.1 packet shaping: ICMP[RAW using context queue]
Wrote 28 byte ICMP packet from context "echo 9"; check the wire.
Wrote 28 byte ICMP packet from context "echo 8"; check the wire.
Wrote 28 byte ICMP packet from context "echo 7"; check the wire.
Wrote 28 byte ICMP packet from context "echo 6"; check the wire.
Wrote 28 byte ICMP packet from context "echo 5"; check the wire.
Wrote 28 byte ICMP packet from context "echo 4"; check the wire.
Wrote 28 byte ICMP packet from context "echo 3"; check the wire.
Wrote 28 byte ICMP packet from context "echo 2"; check the wire.
Wrote 28 byte ICMP packet from context "echo 1"; check the wire.
Wrote 28 byte ICMP packet from context "echo 0"; check the wire.
```

Wi-Fi (icmp)에서 캡처 중

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

표시 필터 적용 ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Len	Info
15	0.002142	192.168.0.1	192.168.0.10	ICMP	52	Echo (ping) reply id=0x0042, seq=66.
16	0.002142	192.168.0.1	192.168.0.10	ICMP	52	Echo (ping) reply id=0x0042, seq=66.
17	0.007113	192.168.0.1	192.168.0.10	ICMP	52	Echo (ping) reply id=0x0042, seq=66.
18	0.007300	192.168.0.1	192.168.0.10	ICMP	52	Echo (ping) reply id=0x0042, seq=66.
19	0.007699	192.168.0.1	192.168.0.10	ICMP	52	Echo (ping) reply id=0x0042, seq=66.
20	0.007699	192.168.0.1	192.168.0.10	ICMP	52	Echo (ping) reply id=0x0042, seq=66.

> Frame 1: 42 bytes on wire (336 bits), 42 bytes captured
> Ethernet II, Src: 14:18:c3:7e:dc:0e, Dst: 70:5d:cc:92:5d:00
> Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.10

Internet Control Message Protocol

- Type: 8 (Echo (ping) request)
- Code: 0
- Checksum: 0xf77b [correct]
- [Checksum Status: Good]
- Identifier (BE): 66 (0x0042)
- Identifier (LE): 16896 (0x4200)
- Sequence Number (BE): 66 (0x0042)
- Sequence Number (LE): 16896 (0x4200)

> [No response seen]

0000 70 5d cc 92 5d ac 14 18 c3 7e dc 0e 08 00 45 00 p
0010 00 1c e8 c4 00 00 3f 01 00 00 c0 a8 00 0a c0 a8 .
0020 00 01 08 00 f7 7b 00 42 00 42 :

http: Wi-Fi: <live capture in progress> 패킷: 20 프로파일: Default



LINUX PF_SOCKET



PF_PACKET

❖ PF_PACKET

- a socket type used on Linux for packet capture
 - allows to access raw packets directly from the network interface
- libpcap uses PF_PACKET also, providing
 - a high-level library that simplifies packet capturing
 - a cleaner API to interact with various network interfaces

PF_PACKET

❖ Usage procedure (1/2)

■ socket creation

```
int sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
```

■ interface binding

```
struct ifreq ifr;  
struct sockaddr_ll sll;  
  
strncpy(ifr.ifr_name, "eth0", IFNAMSIZ); // set interface name  
res = ioctl(sock, SIOCGIFINDEX, &ifr) ;  
  
memset(&sll, 0, sizeof(sll));  
sll.sll_family = AF_PACKET;  
sll.sll_ifindex = ifr.ifr_ifindex; // 인터페이스 인덱스  
sll.sll_protocol = htons(ETH_P_ALL);  
  
bind(sock, (struct sockaddr *)&sll, sizeof(sll))
```



PF_PACKET

❖ Usage procedure (2/2)

▪ packet generation

```
unsigned char *packet;  
  
int len = sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&sll, sizeof(sll));
```

▪ packet reception

```
unsigned char buffer[2048];  
  
int len = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
```

PF_PACKET

❖ Example (1/3)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <net/if.h>
#include <linux/if_packet.h>
#include <netinet/ether.h>
#include <arpa/inet.h>
#include <sys/ioctl.h>

int main() {
    int sock;
    struct ifreq ifr;
    struct sockaddr_ll sll;
    unsigned char buffer[2048];
```

```
// 소켓 생성
sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if (sock == -1) {
    perror("Socket creation failed");
    return -1;
}

// 네트워크 인터페이스 가져오기
strncpy(ifr.ifr_name, "ens33", IFNAMSIZ);
if (ioctl(sock, SIOCGIFINDEX, &ifr) == -1) {
    perror("Failed to get interface index");
    close(sock);
    return -1;
}

// 소켓 주소 설정
memset(&sll, 0, sizeof(sll));
sll.sll_family = AF_PACKET;
sll.sll_ifindex = ifr.ifr_ifindex;
sll.sll_protocol = htons(ETH_P_ALL);
```


PF_PACKET

❖ Example (2/3)

```
if (bind(sock, (struct sockaddr *)&sll, sizeof(sll)) == -1) {
    perror("Bind failed");
    close(sock);
    return -1;
}

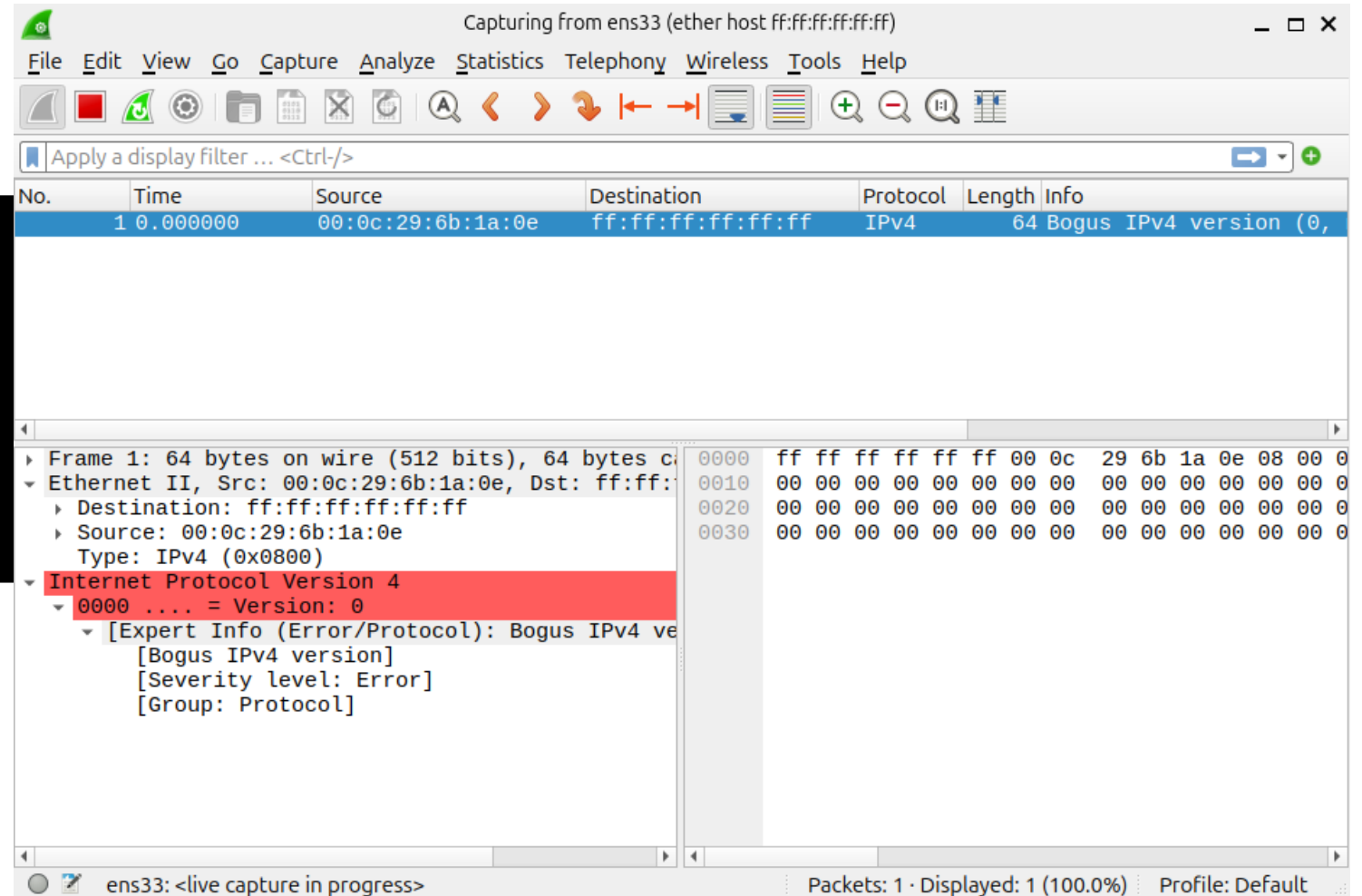
unsigned char packet[64] = {
    0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, // 목적지 MAC (브로드캐스트)
    0x00, 0x0C, 0x29, 0x6B, 0x1A, 0x0E, // 출발지 MAC
    0x08, 0x00,                          // 이더넷 타입 (IPv4)
};

// 패킷 수신
int len = sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&sll, sizeof(sll));
printf("Packet sent successfully\n");
}

// 패킷 수신
printf("Listening for packets...\n");
int len = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
printf("Received packet of length %d\n", len);
close(sock);
return 0;
}
```

❖ Example (3/3)

```
$ gcc pf_ex.c
$ sudo ./a.out
Packet sent successfully
Listening for packets...
```

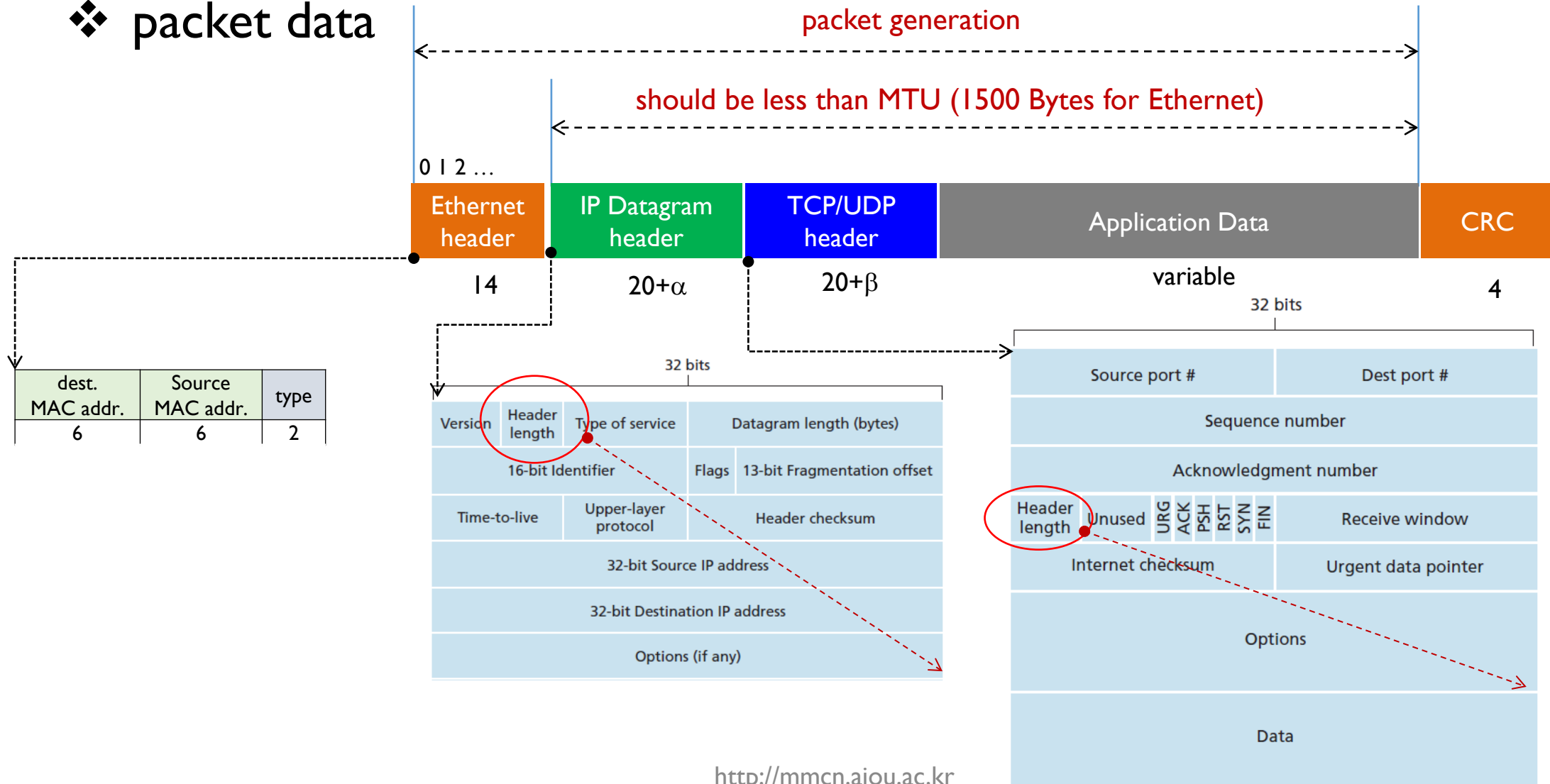


LIBPCAP PACKET GENERATION EXAMPLE



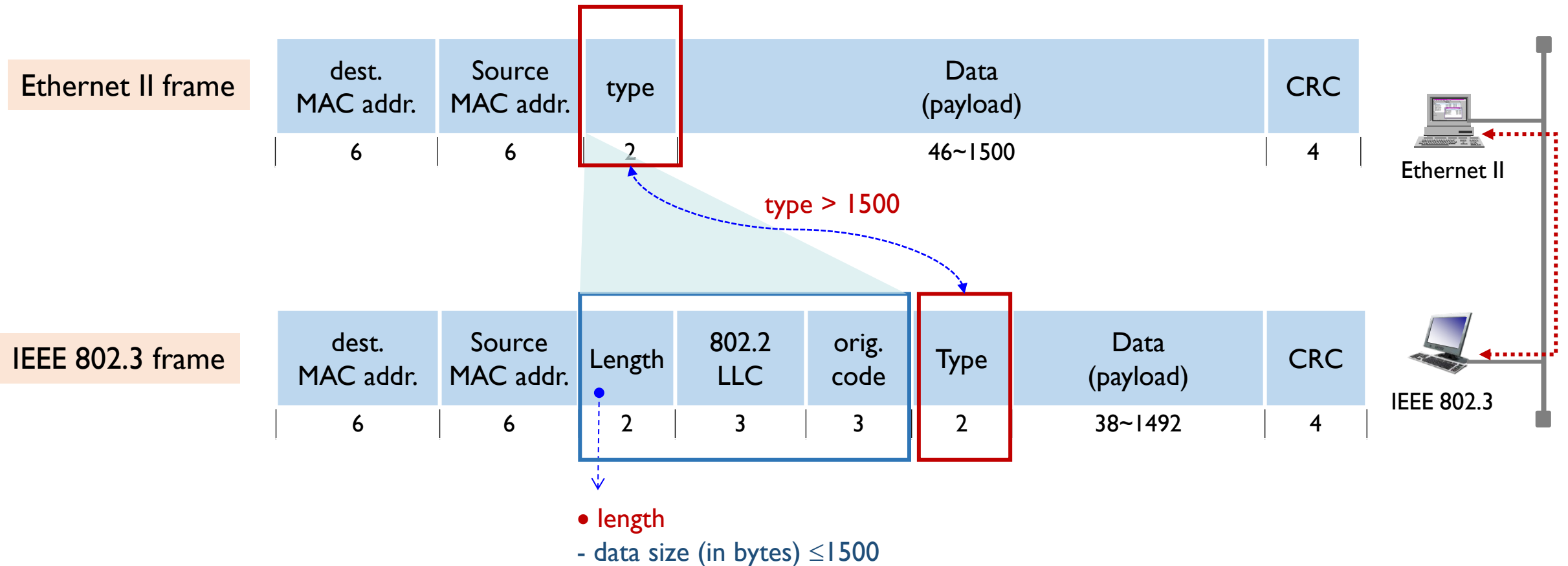
Packet Generation Data Structure

❖ packet data



Ethernet Header Generation (1/3)

❖ Ethernet II and IEEE 802.3 Ethernet frame structure



Ethernet Header Generation (2/3)

❖ Ethernet header data structure

- by referring to `/usr/include/linux/if_ether.h` in linux

```
#define ETH_ALEN      6           /* Octets in one ethernet addr  */
#define ETH_P_IP      0x0800     /* Internet Protocol packet      */
#define ETH_P_ARP      0x0806     /* Address Resolution packet     */

struct ethhdr {
    unsigned char    h_dest [ETH_ALEN];   /* destination eth addr */
    unsigned char    h_source [ETH_ALEN]; /* source ether addr    */
    unsigned short   h_proto;              /* packet type ID field */
};
```

h_dest	h_source	h_proto		
dest. MAC addr.	Source MAC addr.	type	Data (payload)	CRC
6	6	2	46~1500	4

Ethernet Header Generation (3/3)

❖ Example code to generate Ethernet header

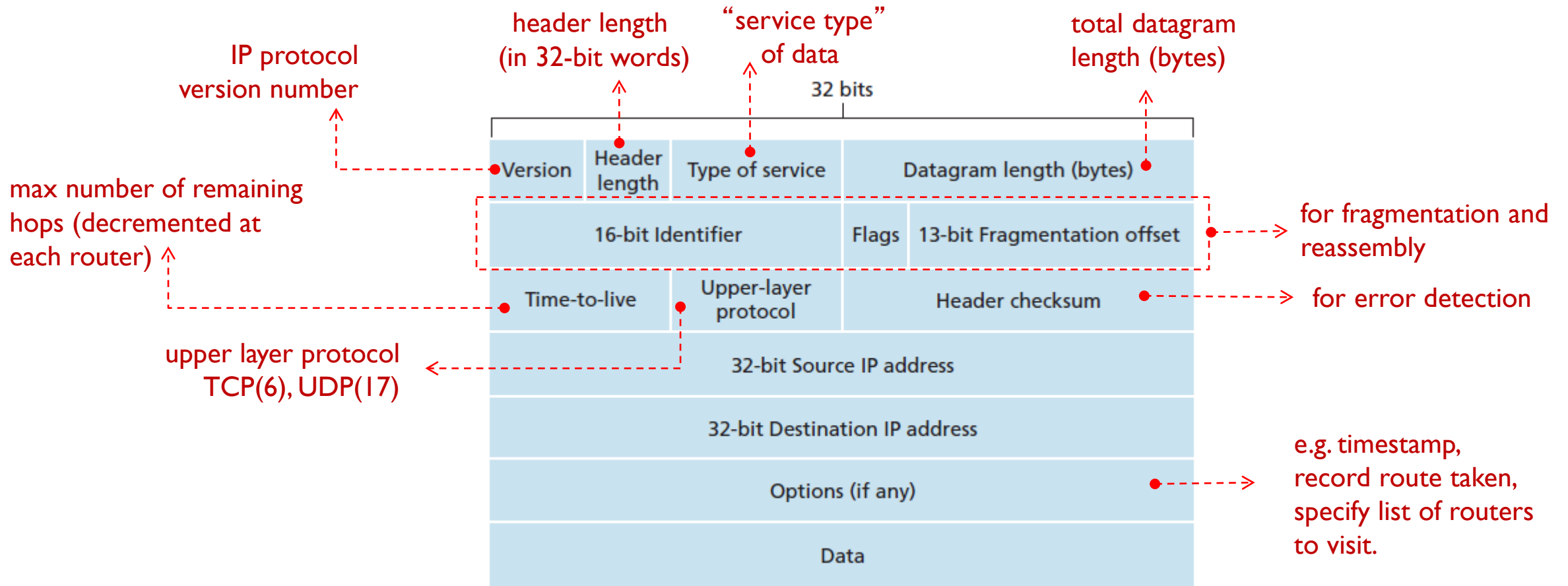
```
struct ethhdr *eh;
...
// destination MAC: e.g. 08:01:01:01:AF:C0
eh->h_dest [0] = 0x08;
eh->h_dest [1] = 0x02;
eh->h_dest [2] = 0x01;
eh->h_dest [3] = 0x01;
eh->h_dest [4] = 0xAF;
eh->h_dest [5] = 0xC0;

// source MAC: e.g. 08:02:02:02:AC:02
eh->h_source [0] = 0x08;
eh->h_source [1] = 0x02;
eh->h_source [2] = 0x02;
eh->h_source [3] = 0x02;
eh->h_source [4] = 0xAC;
eh->h_source [5] = 0x02;

// type field: "#define      ETH_P_IP      0x0800"
eh->h_proto = htons(ETH_P_IP);
```

IP Header Generation (1/3)

❖ IP datagram header format



IP Header Generation (2/3)

- ❖ IP datagram header data structure
 - by referring to `/usr/include/linux/ip.h` in linux

```
struct iphdr
{
    unsigned int    version:4;    // big_endian
    unsigned int    ihl:4;        // big_endian
    unsigned char    tos;
    unsigned short   tot_len;
    unsigned short   id;
    unsigned short   frag_off;
    unsigned char    ttl;
    unsigned char    protocol;
    unsigned short   check;
    unsigned int     saddr;
    unsigned int     daddr;
    /*The options start here. */
};
```

IP Header Generation (3/3)

❖ Example code to generate IP header

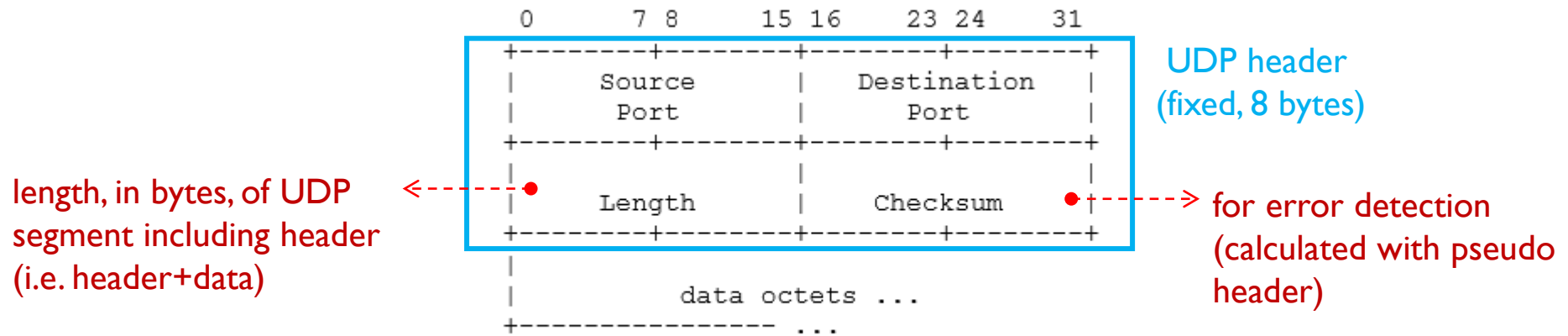
```
struct iphdr *iph;
...

/* IP Header */
iph->version      = 4;
iph->.ihl          = 5;                // no option
iph->tos           = 0;
iph->tot_len       = htons(totlen);    // iphdr + transhdr + data
iph->id            = htons(54321);
iph->frag_off      = 0;                // no fragmentation
iph->ttl           = 255;               // hops
iph->protocol      = IPPROTO_UDP;      // upper layer
iph->check         = 0;                // initially 0
iph->saddr         = inet_addr("192.168.0.11"); // source IP address
iph->daddr         = inet_addr("192.168.0.22"); // destination IP address

/* Calculate IP checksum on completed header */
iph->check = checksum((u_short *)iph, sizeof(struct iphdr));
```

UDP Header Generation (1/3)

❖ UDP header format

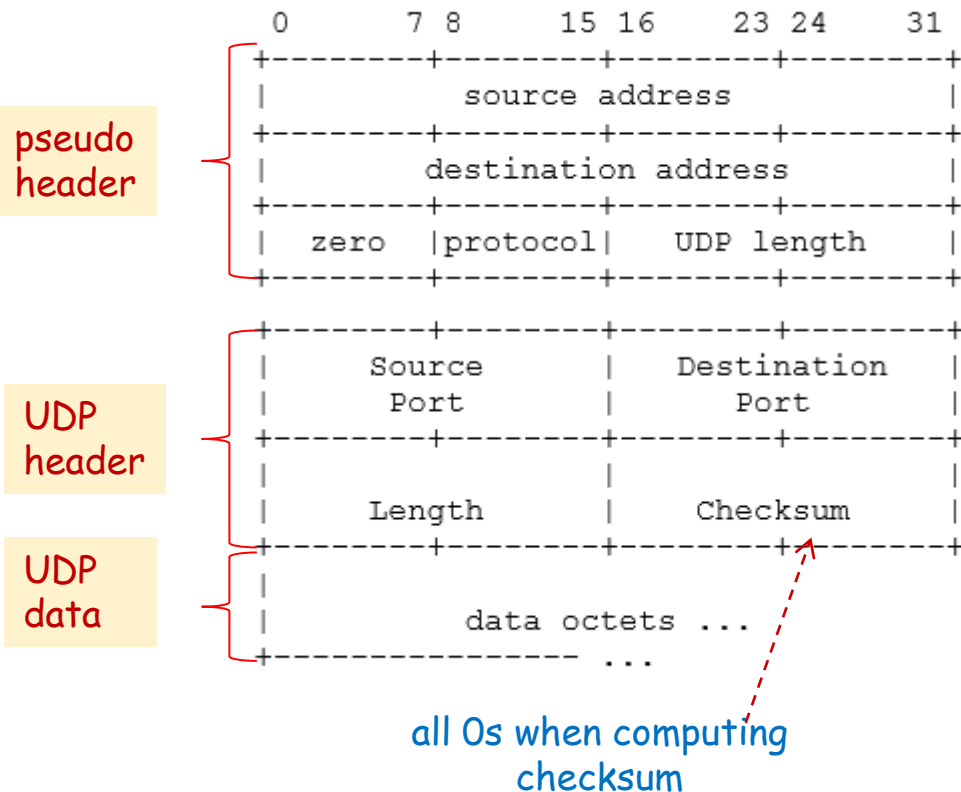


■ data structure

```
struct udphdr
{
    u_int16_t source; /* source port */
    u_int16_t dest;   /* destination port */
    u_int16_t len;     /* udp length */
    u_int16_t check;   /* udp checksum */
};
```

UDP Header Generation (2/3)

❖ UDP pseudo header for checksum calculation



```
struct pseudo_header    //for checksum calculation
{
    u_int32_t    source_address; // source IP address
    u_int32_t    dest_address;   // dest. IP address
    u_int8_t     placeholder;    // all zeros
    u_int8_t     protocol;       // upper layer protocol
                                   // at IP layer
    u_int16_t    length;         // length of transport
                                   // layer header

    struct udphdr trnsh;        // trans. layer header
    unsigned char data[4096];   // data at app. layer
};
```

UDP Header Generation (3/3)

❖ example code to generate UDP header

```
struct iphdr      *iph;
struct udphdr     *udph;
struct pseudo_header psh;

/* UDP header */
udph->source      = htons(sport);          // saddr.sin_port
udph->dest        = htons(dport);          // daddr.sin_port
udph->len          = htons(8+ dlen);        // 8: udp header length, dlen:
udph->check        = 0;                    // initially 0

/* UDP pseudo header */
psh.source_address = iph->saddr; // source IP address
psh.dest_address  = iph->daddr; // destination IP address
psh.placeholder    = 0;
psh.protocol      = iph->protocol; // upper layer protocol (UDP)
psh.length         = udph->len;    // udp length

/* Calculate IP checksum on completed header */
memcpy(&psh.trnsh, udph, sizeof (struct udphdr));
if ( udata != NULL ) memcpy(&psh.data, udata, sizeof(udata));
udph->check          = checksum((u_short *)&psh, sizeof(struct pseudo_header));
```

UDP – Checksum

❖ C code for UDP checksum calculation (1/2)

```
u16 udp_sum_calc (u16 len_udp, u16 src_addr[], u16 dest_addr[], BOOL padding, u16 buff[])
{
    u16 prot_udp=17;
    u16 padd=0;
    u16 word16;
    u32 sum;

    // Find out if the length of data is even or odd number. If odd,
    // add a padding byte = 0 at the end of packet
    if (padding&1==1){
        padd=1;
        buff[len_udp]=0;
    }

    //initialize sum to zero
    sum=0;

    // make 16 bit words out of every two adjacent 8 bit words and
    // calculate the sum of all 16 bit words
    for (i=0; i<len_udp+padd; i=i+2){
        word16 = ((buff[i]<<8)&0xFF00)+(buff[i+1]&0xFF);
        sum = sum + (unsigned long)word16;
    }
}
```

UDP – Checksum

❖ C code for UDP checksum calculation (2/2)

```
// add the UDP pseudo header which contains
// the IP source and destination addresses
for (i=0;i<4;i=i+2){
    word16 =((src_addr[i]<<8)&0xFF00)+(src_addr[i+1]&0xFF);
    sum=sum+word16;
}

for (i=0;i<4;i=i+2){
    word16 =((dest_addr[i]<<8)&0xFF00)+(dest_addr[i+1]&0xFF);
    sum=sum+word16;
}

// the protocol number and the length of the UDP packet
sum = sum + prot_udp + len_udp;

// keep only the last 16 bits of the 32 bit calculated sum and add the carries
while (sum>>16)
    sum = (sum & 0xFFFF)+(sum >> 16);

// Take the one's complement of sum
sum = ~sum;

return ((u16) sum);
}
```

UDP Packet Generation – All Together (1/2)

```
/* useful variables */
char          sendbuf[4096];
struct ethhdr *eh;
struct iphdr  *iph;
struct udphdr *udph;
char          *udata;
struct pseudo_header psh;
int           pk_len;

...
/* Retrieve the device list */
res = pcap_findalldevs(&alldevs, errbuf);
...

/* Open the adapter */
adhandle= pcap_open_live(d->name, SNAPLEN, 1, 1000, errbuf );

/* Prepare headers*/
eh      = (struct ethhdr*) sendbuf;
iph     = (struct iphdr*) (sendbuf+sizeof(struct ethhdr));
udph    = (struct udphdr*) (sendbuf+sizeof(struct ethhdr)+sizeof(struct iphdr));
udata   = (char*) (sendbuf+sizeof(struct ethhdr)+sizeof(struct iphdr) + sizeof(struct udphdr));
```


UDP Packet Generation – All Together (2/2)

```
/* Ethernet header */
... see Ethernet header generation

/* IP header */
... see IP header generation (length & check later)

/* UDP header */
... see UDP header generation

/* Data */
udata = ...

/* Fill remaining header fields */
iph->tot_len = ...;
iph->check   = checksum (...);
udph->len    = ...;
udph->check  = checksum (...);

/* send the frame to the interface */
pk_len = sizeof(struct ethhdr)+sizeof(struct iphdr)+sizeof(struct udphdr)+ strlen(udata);
pcap_inject (adhandle, sendbuf, pk_len);
```

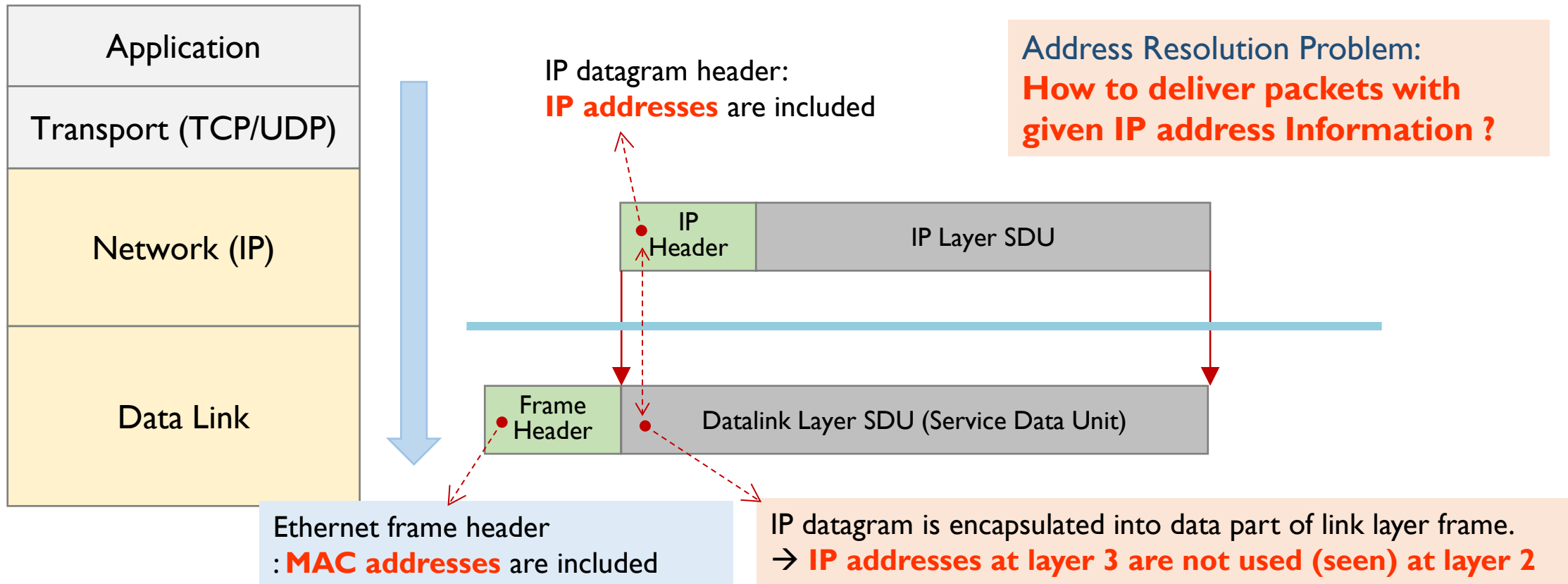


ARP EXAMPLE

ARP (Address Resolution Protocol)

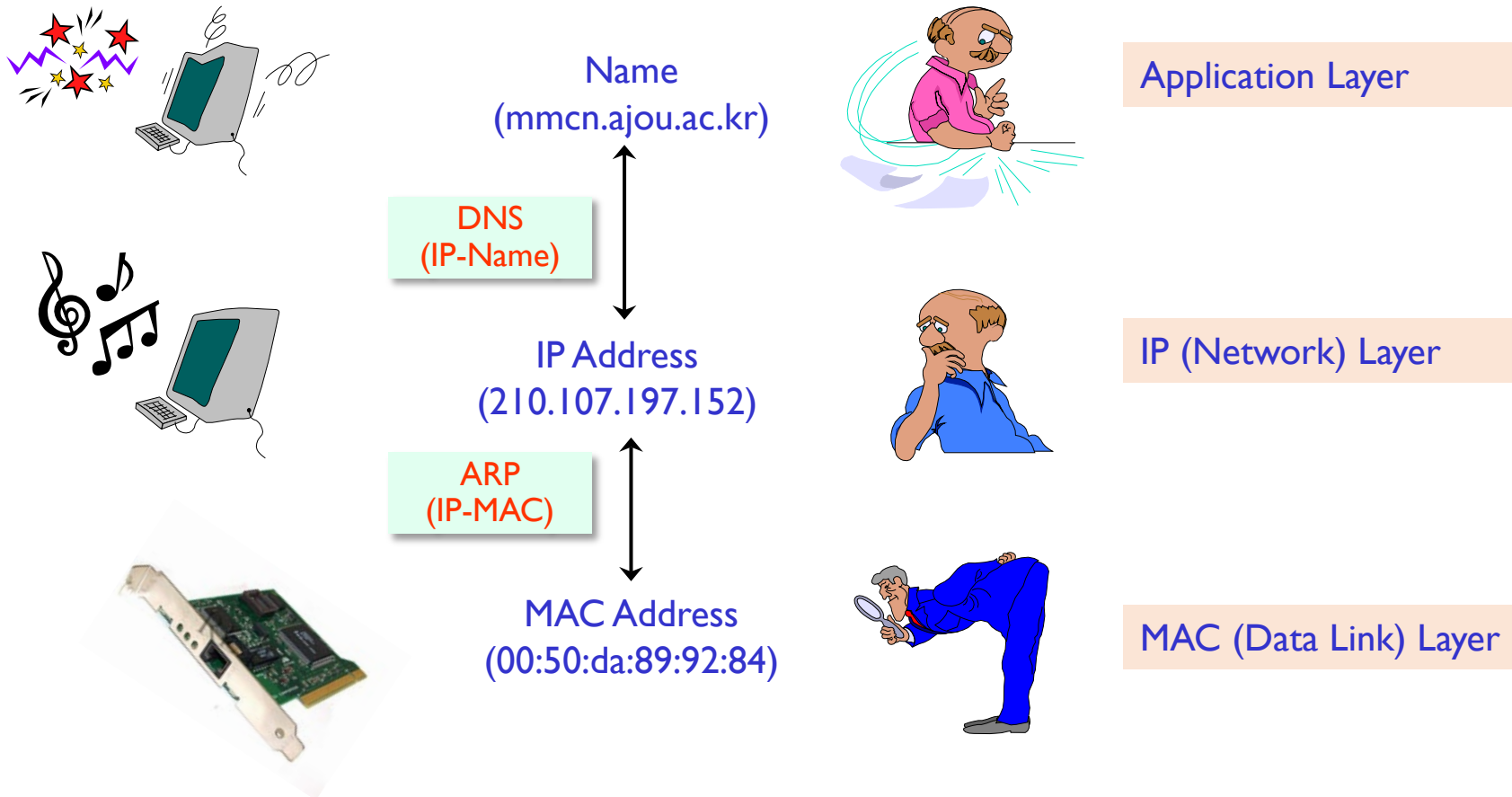
❖ Address Resolution Problem

- IP address at layer 3 is encapsulated (not seen) at datalink layer



ARP

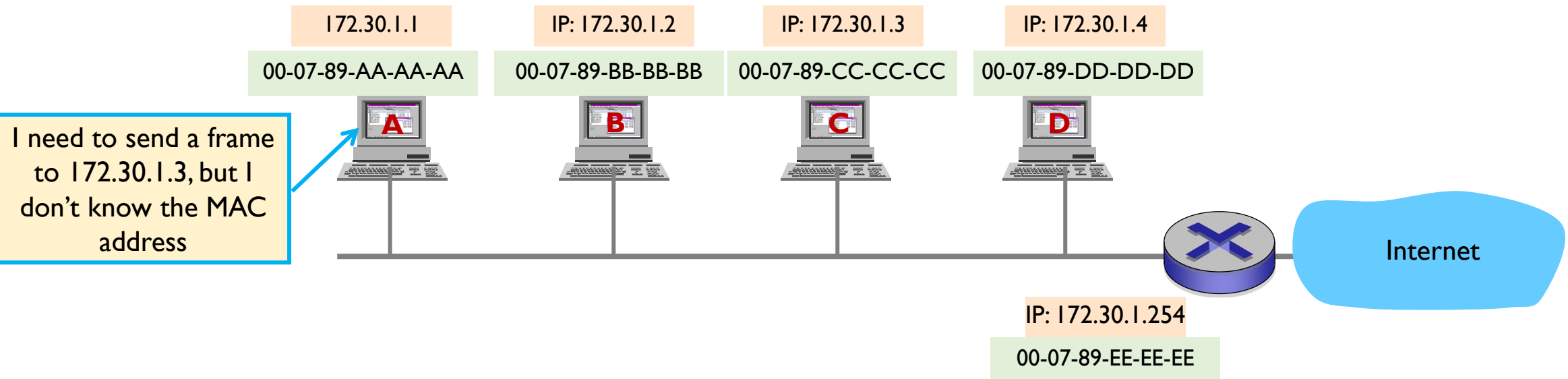
❖ Resolution Protocols for Name and Address Conversion



ARP Operation

❖ ARP Operation Environments

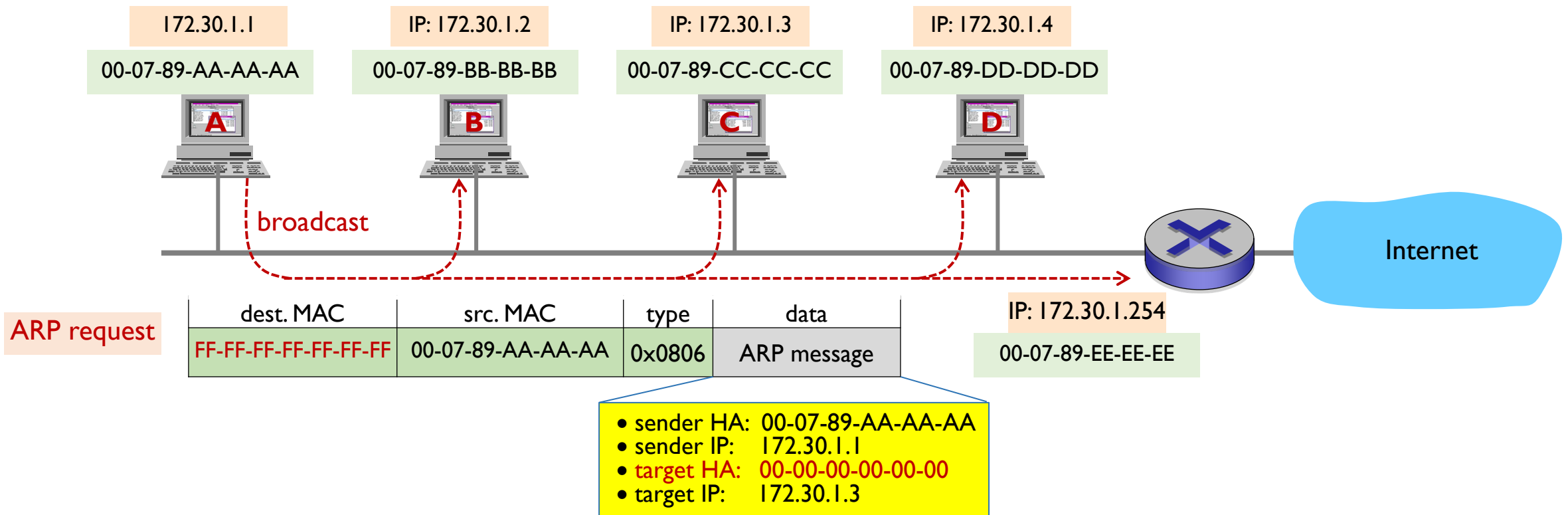
- to send a frame on a LAN media,
 - hosts must know destination MAC address
- host A wants to deliver a frame to host C
- host A knows
 - only host C's IP address, but, host C's MAC address is not known)



ARP Operation

❖ ARP Request

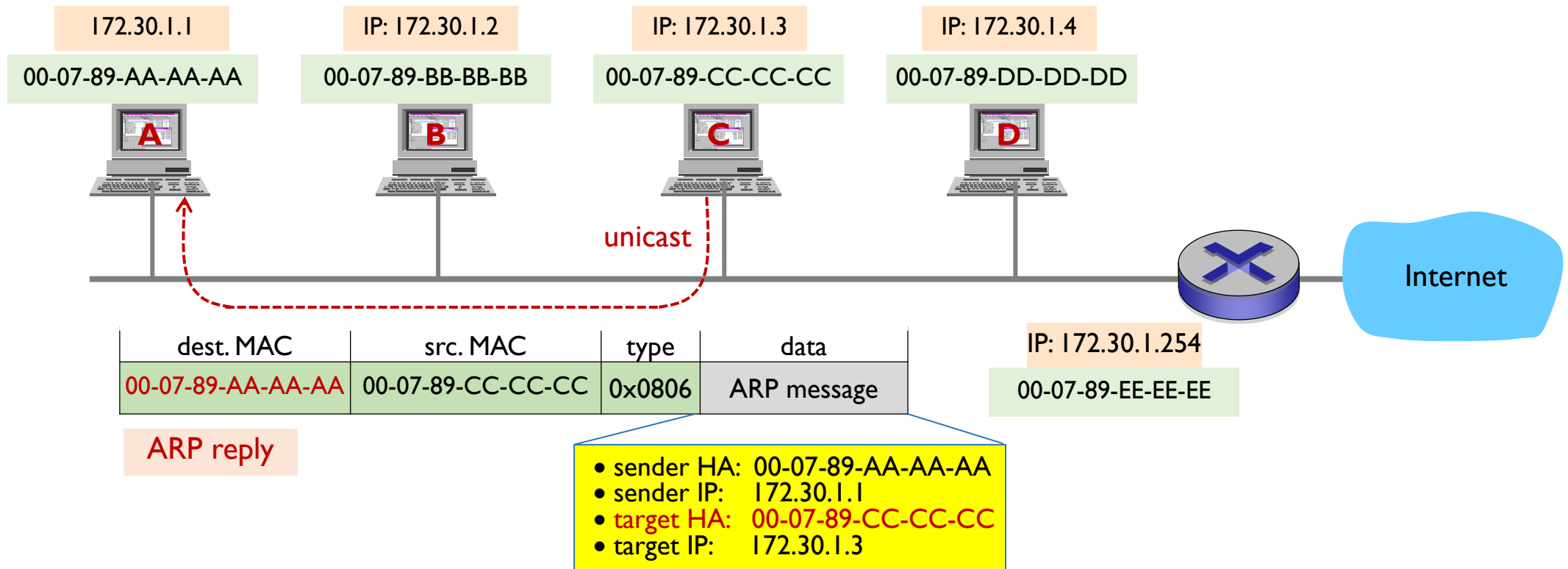
- host A broadcasts ARP Request message to all hosts in its network,
 - in which host C's IP address is included



ARP Operation

❖ ARP Reply

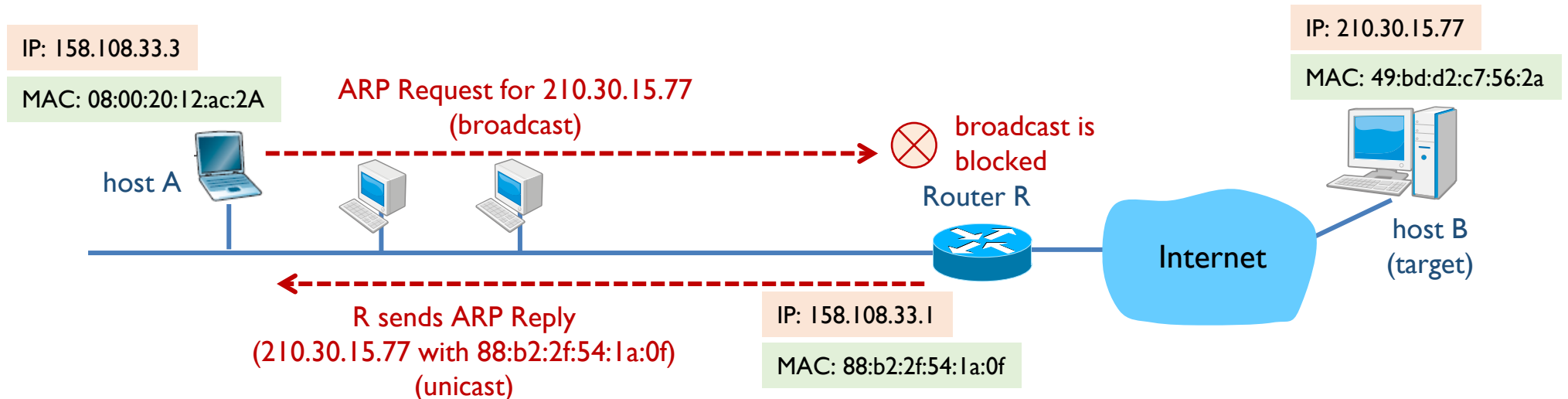
- host C – answers ARP reply message with its MAC address
- host B, C, and D – keep host A's IP/MAC addresses in its ARP table for future use



Proxy ARP

❖ Proxy ARP

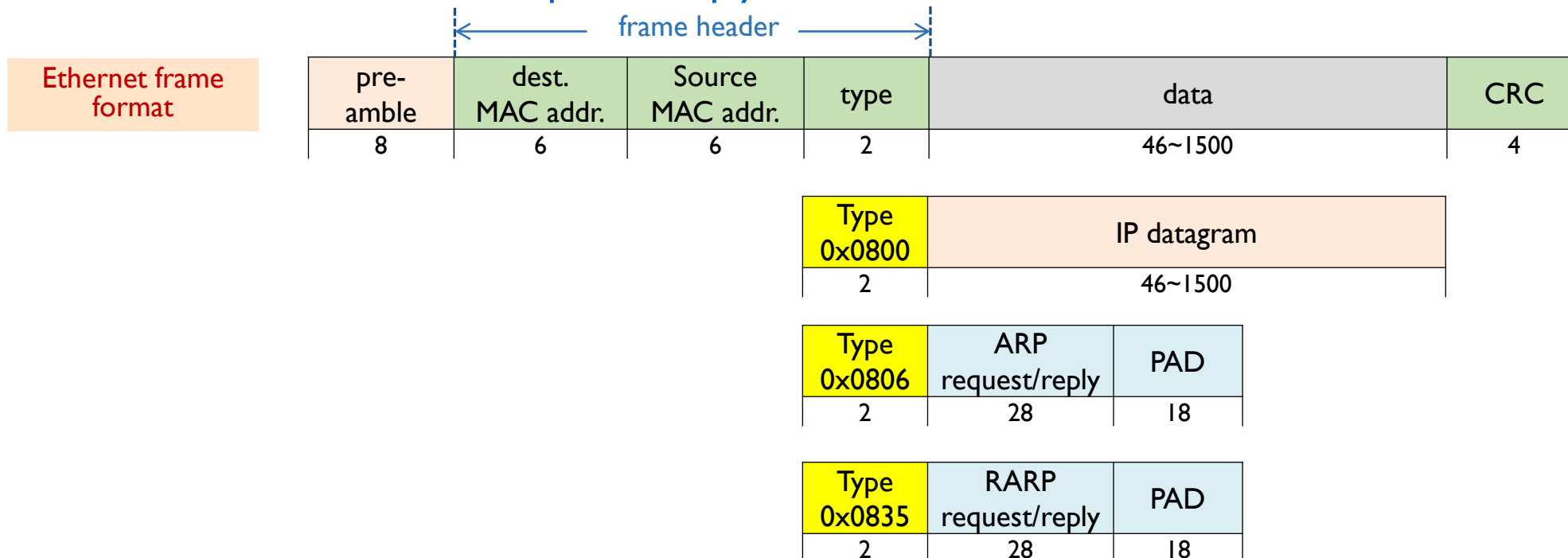
- Router blocks broadcasted ARP Request message
- Proxy ARP allows
 - the router to respond for remote host outside the local network
 - example
 - host A wants to communicate with host B, but knows only B's IP address



ARP Message Format

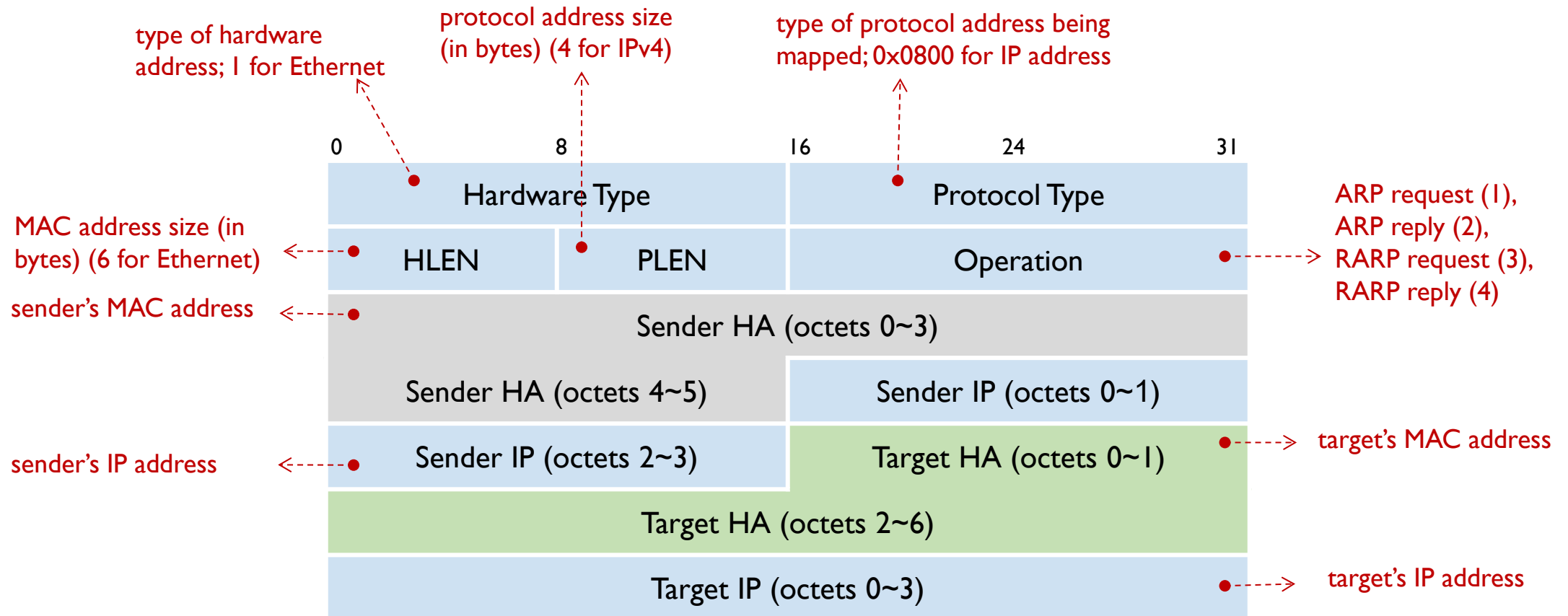
❖ Ethernet encapsulation

- type field
 - 0x0800 : IP datagram
 - 0x0806 : ARP Request / Reply
 - 0x0835 : RARP Request / Reply



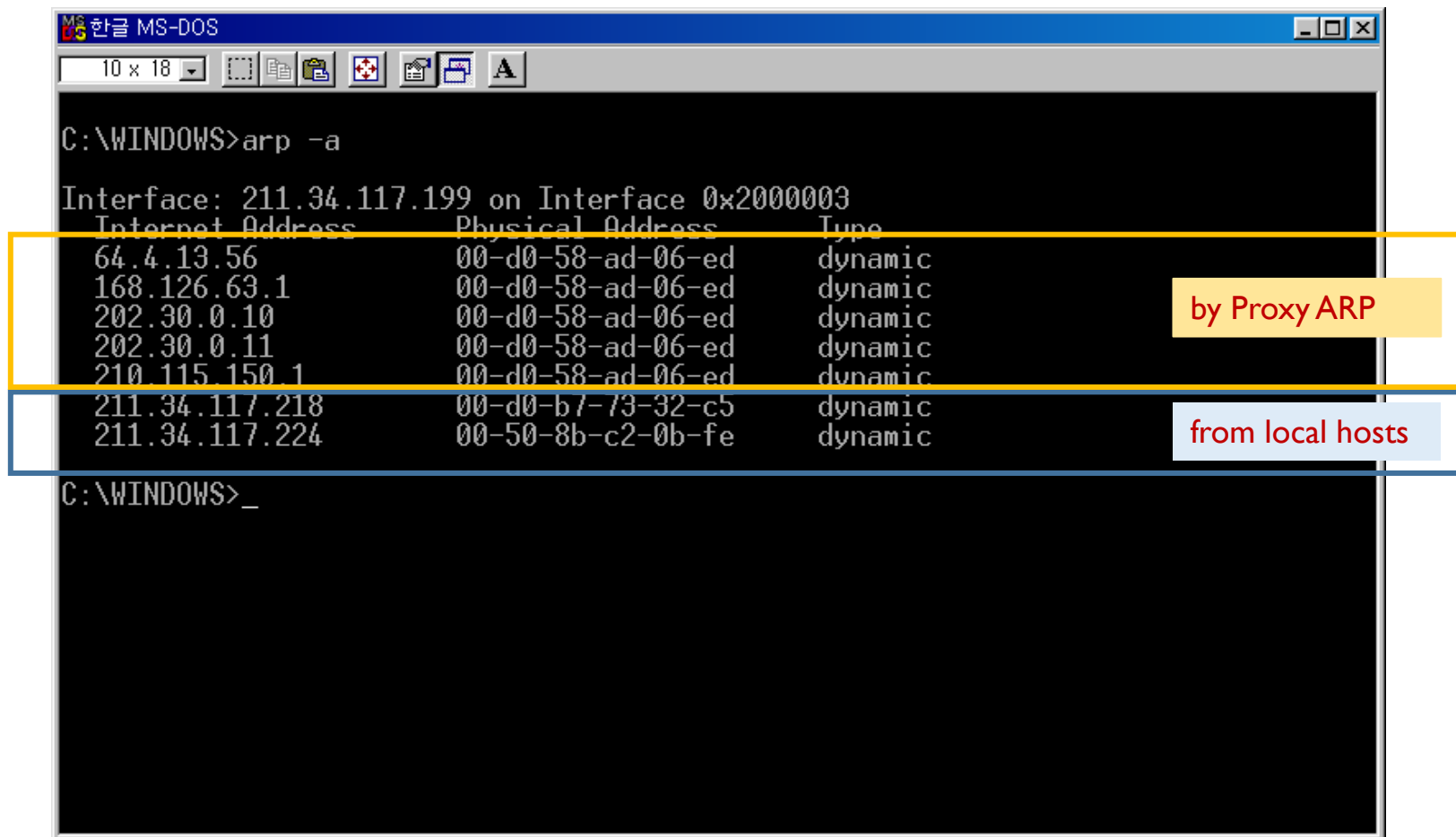
ARP Message Format

❖ ARP Message Format



ARP Caching

❖ ARP cache



```
MS 한글 MS-DOS
10 x 18
C:\WINDOWS>arp -a

Interface: 211.34.117.199 on Interface 0x2000003
Internet Address      Physical Address      Type
64.4.13.56            00-d0-58-ad-06-ed    dynamic
168.126.63.1          00-d0-58-ad-06-ed    dynamic
202.30.0.10           00-d0-58-ad-06-ed    dynamic
202.30.0.11           00-d0-58-ad-06-ed    dynamic
210.115.150.1         00-d0-58-ad-06-ed    dynamic
211.34.117.218        00-d0-b7-73-32-c5    dynamic
211.34.117.224        00-50-8b-c2-0b-fe    dynamic

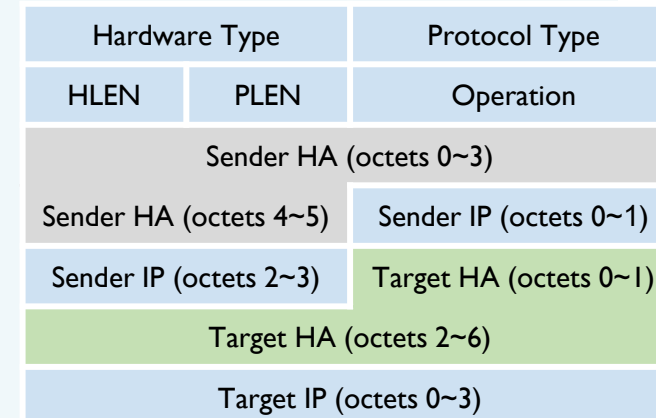
C:\WINDOWS>_
```

by Proxy ARP

from local hosts

Example Code – ARPgeneration.c (1/7)

```
...
#define ETH_ALEN      6           // Octets in one ethernet addr
#define IP_ALEN       4           // Octets of IP address
#define ETH_P_ARP     0x0806     // Address Resolution packet
#define ARP_LEN       60         // ARP Message Length
...
struct arpmsg {
    unsigned short h_type;        // Hardware Type
    unsigned short p_type;        // Protocol Type
    unsigned char h_len;          // Hardware address length
    unsigned char p_len;          // Protocol address length
    unsigned short opcode;        // Operation
    unsigned char sender_ha[ETH_ALEN]; // Sender Hardware Address
    unsigned char sender_ip[IP_ALEN]; // Sender IP Address
    unsigned char target_ha[ETH_ALEN]; // Target Hardware Address
    unsigned char target_ip[IP_ALEN]; // Target IP Address
};
...
unsigned char my_mac[ETH_ALEN] = { 0x98, 0x5F, 0xD3, 0x4A, 0xD4, 0x7 };
unsigned char dst_mac[ETH_ALEN] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };
unsigned char my_ip[IP_ALEN] = { 172, 30, 1, 40 };
unsigned char tgt_ip[IP_ALEN] = { 172, 30, 1, 21 };
```



- it should be your interface's MAC address

- it should be your interface's IP address

- it should be your target IP address

Example Code – ARPgeneration.c (2/7)

```
int main()
{
    u_char    msg[2000];
    int       offset=0;
    ...
    pcap_findalldevs(&alldevs, errbuf); // Retrieve the device list
    ...
    adhandle= pcap_open_live(d->name, SNAPLEN, 1, 1000, errbuf); // Select the capture device
    ...
    pcap_freealldevs(alldevs);
    ...
    // build raw ARP message
    memset(msg, 0, sizeof(msg));
    build_Ethernet_Header(msg, ETH_P_ARP, &offset);           // Ethernet Header
    build_ARP_Message(msg, &offset);                          // ARP Message just after Ethernet Header

    // send raw ARP message
    for ( int j=0; j<2; j++ ) pcap_sendpacket(adhandle, msg, ARP_LEN);
    ...
    pcap_close(adhandle);
    return 0;
}
```

Example Code – ARPgeneration.c (3/7)

```
void build_Ethernet_Header (u_char *msg, unsigned short type, int *offset)
{
    struct ethhdr*   eth;
    int              i;

    eth = (struct ethhdr*)msg + *offset;

    // ethernet header
    // destination MAC address
    for (i = 0; i < ETH_ALEN; i++) eth->h_dest[i] = dst_mac[i];

    // source MAC address (6 bytes)
    for (i = 0; i < ETH_ALEN; i++) eth->h_source[i] = my_mac[i];

    // type field
    eth->h_proto = htons(type);

    *offset += sizeof(struct ethhdr);
}
```

Example Code – ARPgeneration.c (4/7)

```
void build_ARP_packet (u_char *msg, int *offset) {
    struct arpmsg* arp;
    int i;

    arp = (struct arpmsg*)(msg + *offset);

    // ARP message
    arp->h_type = htons(0x0001);    // hardware type : ethernet 0x0001
    arp->p_type = htons(0x0800);    // protocol type : IP 0x0800

    arp->h_len = ETH_ALEN;          // hardware address size (in byte)
    arp->p_len = IP_ALEN;            // protocol address size (in byte)

    arp->opcode = htons(0x0001);    // opcode : ARP request 0x0001

    for (i=0; i< ETH_ALEN; i++) arp->sender_ha[i] = my_mac[i];    // sender mac address
    for (i=0; i< IP_ALEN; i++) arp->sender_ip[i] = my_ip[i];    // sender IP address

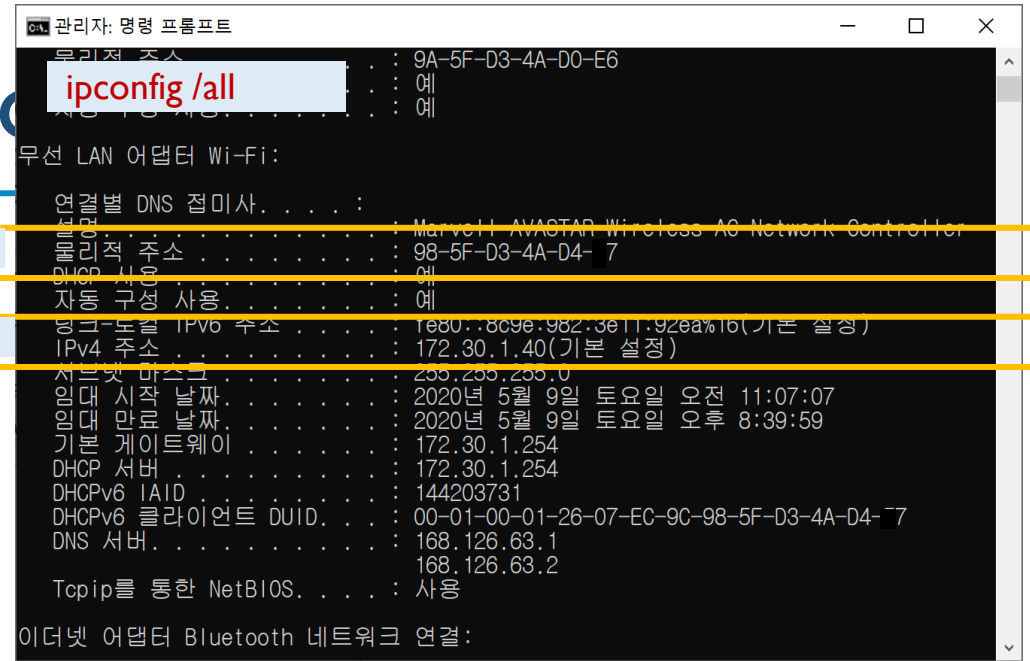
    for (i=0; i< ETH_ALEN; i++) arp->target_ha[i] = 0x00 ;    // target mac address
    for (i=0; i< IP_ALEN; i++) arp->target_ip[i] = tgt_ip[i];    // sender IP address

    *offset += sizeof(struct arpmsg);
}
```

Example Code – ARPgeneration

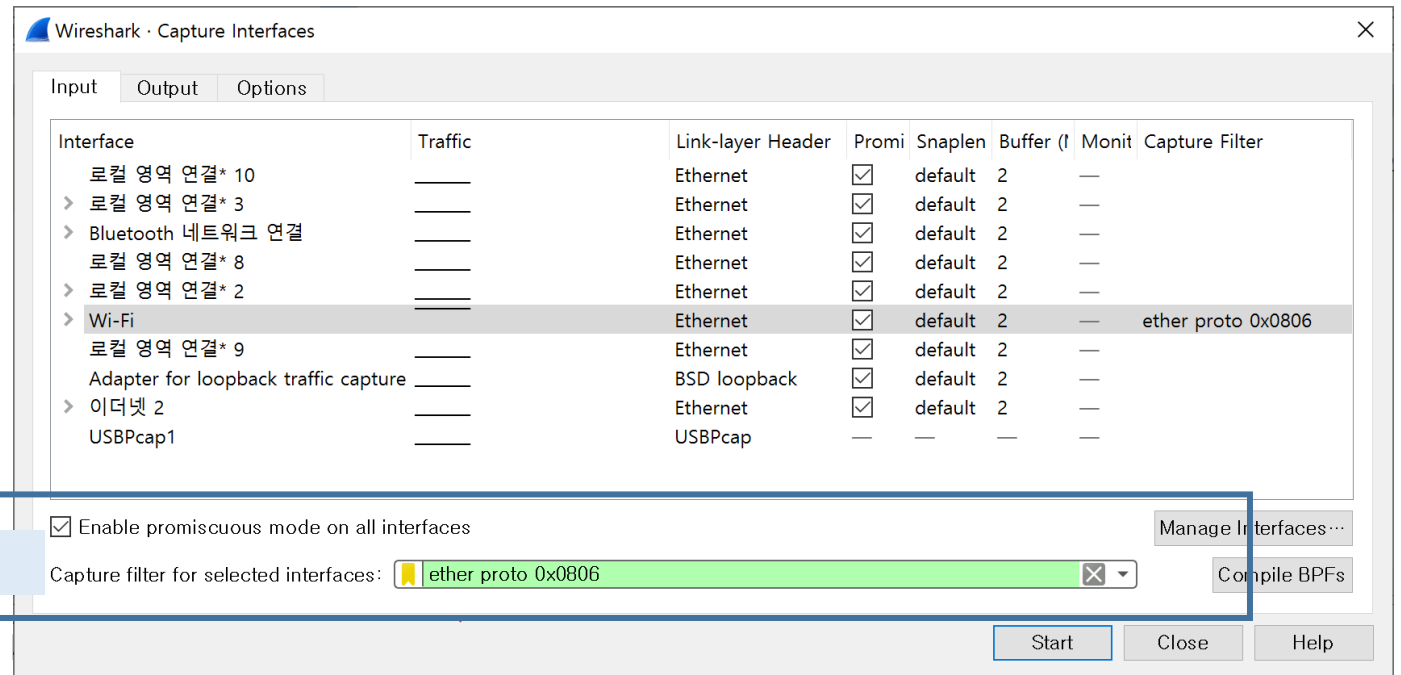
❖ To run the example

- check your ip and mac address
 - use “ipconfig /all” in command prompt
- flush ARP cache
 - “arp -d” in command prompt (as administrator mode)
- set filter in WireShark
 - Capture > Option
 - “ether proto 0x0806” (ARP)
- then
 - capture start (WireShark)
 - run ARPgeneration code



```
관리자: 명령 프롬프트
물리적 주소 . . . . . : 9A-5F-D3-4A-D0-E6
MAC address
물리적 주소 . . . . . : 98-5F-D3-4A-D4-7
IP address
IPv4 주소 . . . . . : 172.30.1.40(기본 설정)
IPv6 주소 . . . . . : fe80::8c9e:982:3e11:92ea%16(기본 설정)
서브넷 마스크 . . . . . : 255.255.255.0
임대 시작 날짜 . . . . . : 2020년 5월 9일 토요일 오전 11:07:07
임대 만료 날짜 . . . . . : 2020년 5월 9일 토요일 오후 8:39:59
기본 게이트웨이 . . . . . : 172.30.1.254
DHCP 서버 . . . . . : 172.30.1.254
DHCPv6 IAID . . . . . : 144203731
DHCPv6 클라이언트 DUID . . . : 00-01-00-01-26-07-EC-9C-98-5F-D3-4A-D4-7
DNS 서버 . . . . . : 168.126.63.1
                  168.126.63.2
Tcpip를 통한 NetBIOS . . . . : 사용

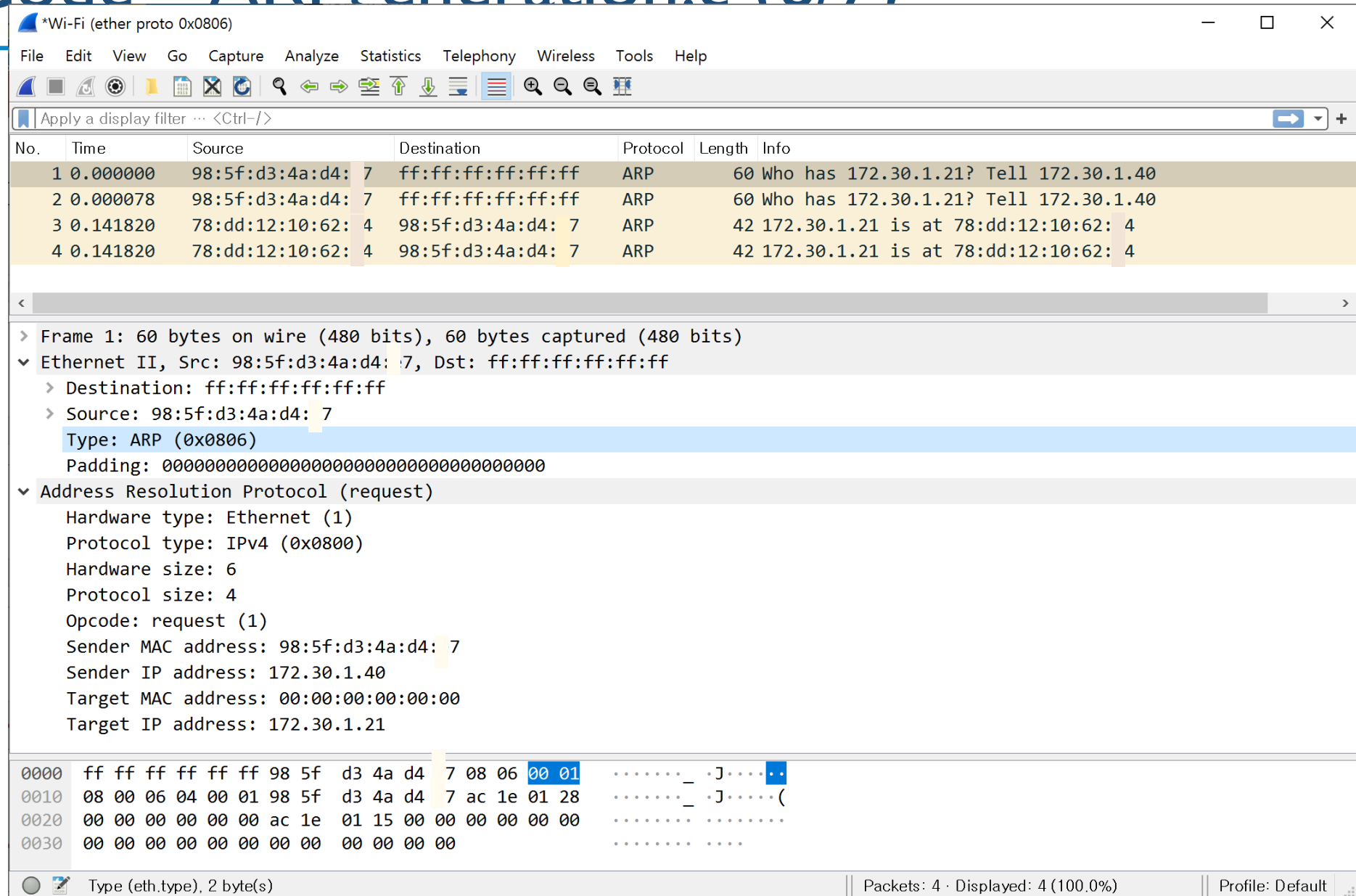
이더넷 어댑터 Bluetooth 네트워크 연결:
```



filter option

Example Code – ARPgeneration.c (6/7)

❖ WireShark



The image shows a Wireshark capture of an ARP request packet. The packet list at the top shows four packets, all of which are ARP requests. The packet details pane shows the structure of the first packet, which is an ARP request (Type: ARP (0x0806)). The packet structure is as follows:

- Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits)
- Ethernet II, Src: 98:5f:d3:4a:d4: 7, Dst: ff:ff:ff:ff:ff:ff
 - Destination: ff:ff:ff:ff:ff:ff
 - Source: 98:5f:d3:4a:d4: 7
 - Type: ARP (0x0806)
 - Padding: 00000000000000000000000000000000
- Address Resolution Protocol (request)
 - Hardware type: Ethernet (1)
 - Protocol type: IPv4 (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - Opcode: request (1)
 - Sender MAC address: 98:5f:d3:4a:d4: 7
 - Sender IP address: 172.30.1.40
 - Target MAC address: 00:00:00:00:00:00
 - Target IP address: 172.30.1.21

The packet bytes pane at the bottom shows the raw data of the packet, with the first 12 bytes highlighted in blue:

```
0000  ff ff ff ff ff ff 98 5f  d3 4a d4 7 08 06 00 01  ..... - .J.... ..
0010  08 00 06 04 00 01 98 5f  d3 4a d4 7 ac 1e 01 28  ..... - .J.....(
0020  00 00 00 00 00 00 ac 1e  01 15 00 00 00 00 00  ..... 
0030  00 00 00 00 00 00 00 00  00 00 00 00  ..... 
```

Example Code – ARPgeneration.c (7/7)

❖ ARP Cache

```
C:\W>arp -a
```

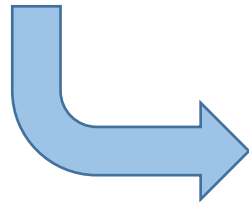
```
인터페이스: 169.254.57.70 --- 0x8  
인터넷 주소      물리적 주소  
224.0.0.22
```

유형
정적

```
인터페이스: 172.30.1.40 --- 0x10  
인터넷 주소      물리적 주소  
172.30.1.254      00-07-89-18-48-6f  
224.0.0.2         01-00-5e-00-00-02  
224.0.0.22        01-00-5e-00-00-16
```

유형
동적

Before Run



After Run

```
C:\W>arp -a
```

```
인터페이스: 169.254.57.70 --- 0x8  
인터넷 주소      물리적 주소  
224.0.0.22  
239.255.255.250
```

유형
정적

```
인터페이스: 172.30.1.40 --- 0x10  
인터넷 주소      물리적 주소
```

```
172.30.1.21      78-dd-12-10-62- 4  
172.30.1.254      00-07-89-18-48-6f  
224.0.0.2         01-00-5e-00-00-02  
224.0.0.22        01-00-5e-00-00-16  
239.255.255.250   01-00-5e-7f-ff-fa
```

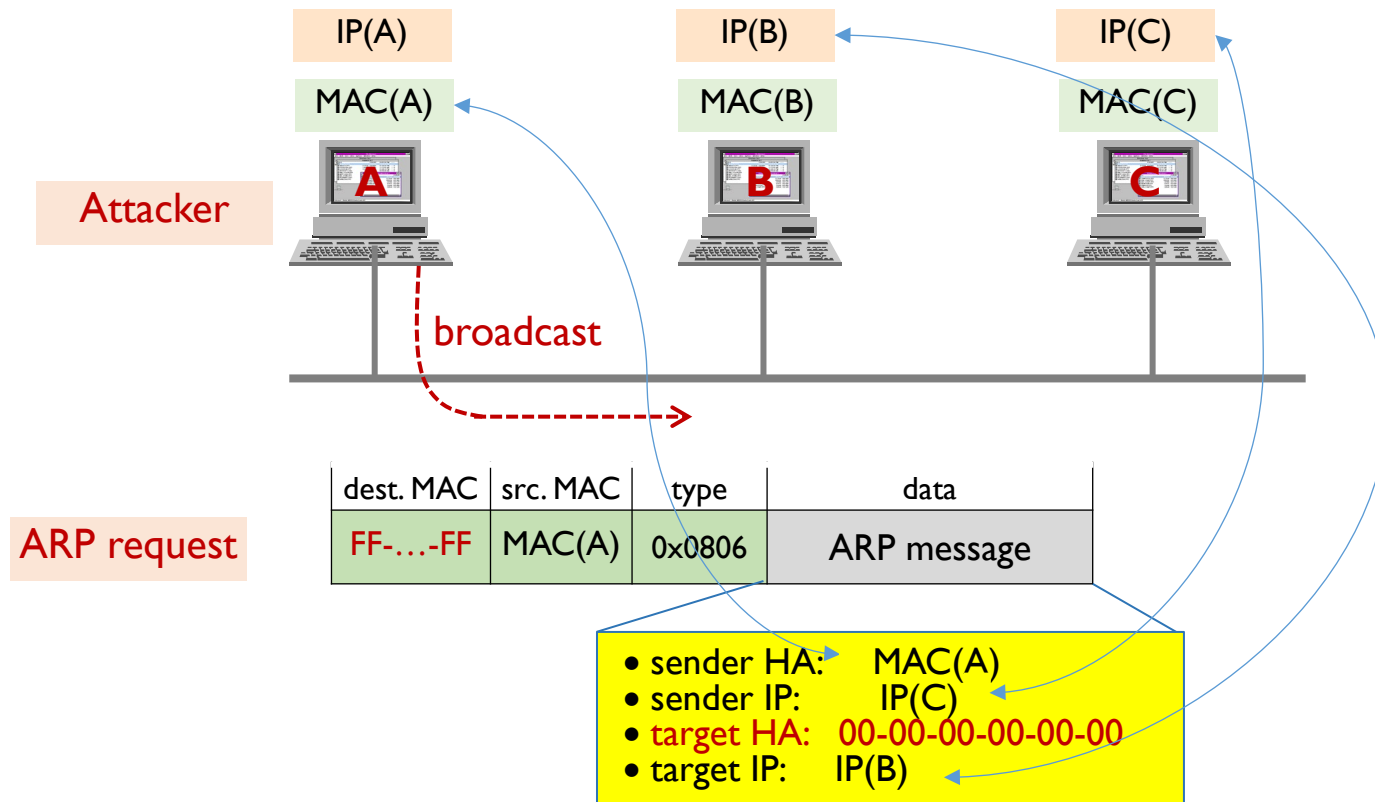
유형
동적



EXAMPLE #2 – ARP POISONING

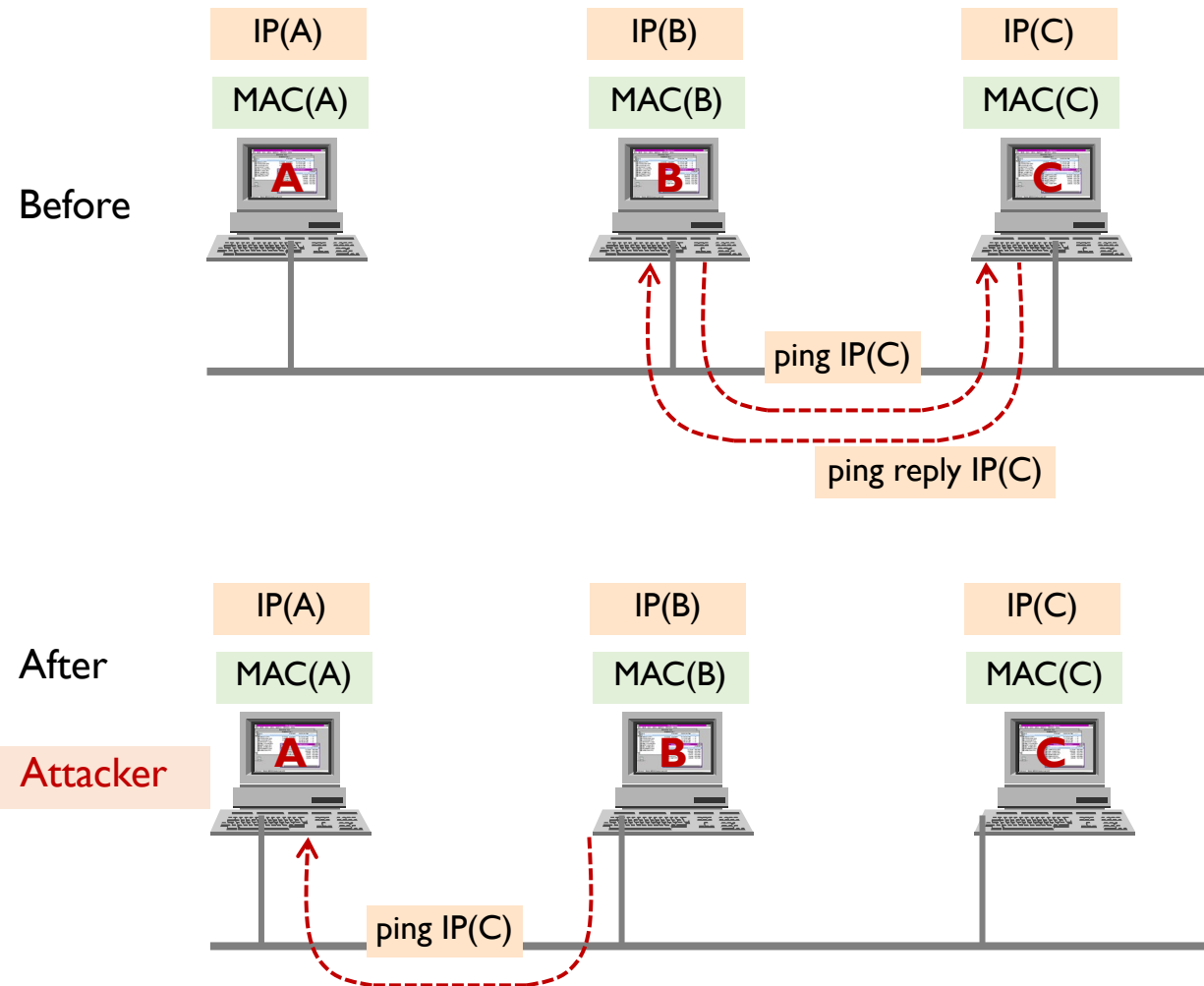
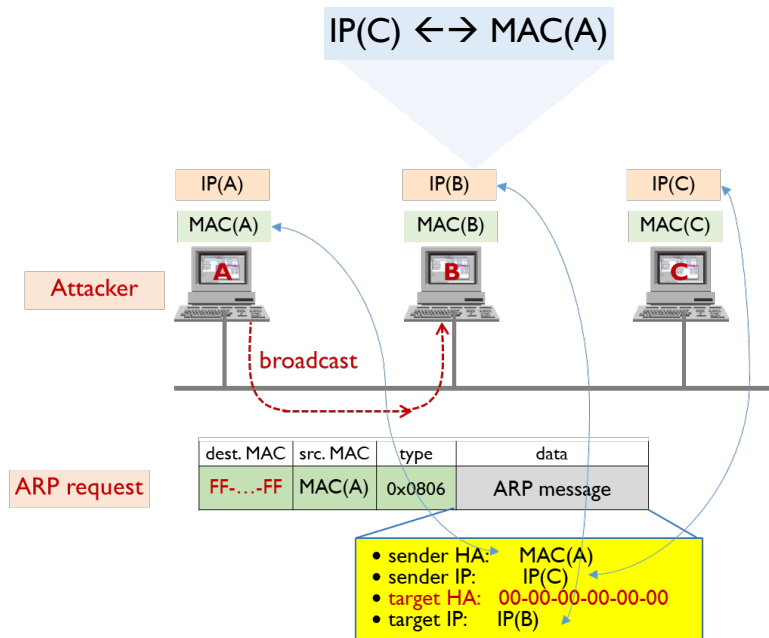
ARP Poisoning

❖ How does it work ?



ARP Poisoning

❖ How does it work ?



Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

